

50 Examples of the 'by' Keyword in Kotlin

The 'by' keyword in Kotlin is used for delegation - both property delegation and class delegation. This document covers various use cases with detailed explanations.

1. Basic Class Delegation

HOW 'by' IS USED: The 'by b' delegates all Base interface methods to the b object. Derived doesn't implement the methods itself - it forwards all calls to b.

```
interface Base {
    fun print()
}

class BaseImpl(val x: Int) : Base {
    override fun print() { println(x) }
}

// 'by b' means: delegate all Base methods to b
class Derived(b: Base) : Base by b

val b = BaseImpl(10)
val derived = Derived(b)
derived.print() // Actually calls b.print()
```

2. Class Delegation with Override

HOW 'by' IS USED: The 'by b' delegates Base methods to b, BUT we can override specific methods. Here printMessage() is overridden while printMessageLine() is still delegated.

```
interface Base {
    fun printMessage()
    fun printMessageLine()
}

class BaseImpl(val x: Int) : Base {
    override fun printMessage() { print(x) }
    override fun printMessageLine() { println(x) }
}

// 'by b' delegates to b, but we override printMessage
class Derived(b: Base) : Base by b {
    override fun printMessage() { print("abc") }
    // printMessageLine is still delegated to b
}
```


3. Lazy Delegation

HOW 'by' IS USED: 'by lazy' delegates the property initialization to a lazy delegate. The lambda is executed only on first access, and the result is cached.

```
// 'by lazy' means: compute value on first access, cache it
val lazyValue: String by lazy {
    println("Computed!")
    "Hello"
}

println(lazyValue) // Computed! Hello (runs lambda)
println(lazyValue) // Hello (uses cached value)
```

4. Lazy with Thread Safety Mode

HOW 'by' IS USED: 'by lazy(mode)' delegates to a lazy delegate with specific thread safety. PUBLICATION mode allows multiple threads to initialize, but only first result is used.

```
// 'by lazy(mode)' specifies thread safety behavior
val lazyValue: String by lazy(LazyThreadSafetyMode.PUBLICATION) {
    expensiveComputation() // Multiple threads may compute
    // but only first result is stored
}
```

5. Lazy with Synchronized Lock

HOW 'by' IS USED: 'by lazy' with SYNCHRONIZED mode (default) uses a lock to ensure only one thread initializes the value.

```
// 'by lazy' with SYNCHRONIZED (default) uses a lock
val lazyValue: String by lazy(LazyThreadSafetyMode.SYNCHRONIZED) {
    "Thread-safe initialization" // Only one thread executes
}
```

6. Lazy with No Lock

HOW 'by' IS USED: 'by lazy' with NONE mode has no synchronization. Faster but not thread-safe - use only for single-threaded contexts.

```
// 'by lazy' with NONE has no thread safety
val lazyValue: String by lazy(LazyThreadSafetyMode.NONE) {
    "Not thread-safe, faster" // Use in single-threaded only
}
```

7. Observable Property

HOW 'by' IS USED: 'by Delegates.observable' delegates to an observable delegate that calls a lambda after each property change, receiving old and new values.

```
import kotlin.properties.Delegates

// 'by Delegates.observable' tracks changes
var name: String by Delegates.observable("Initial") {
    prop, old, new -> // Called after each change
    println("$old -> $new")
}

name = "Updated" // Triggers lambda: Initial -> Updated
```

8. Observable with Custom Logic

HOW 'by' IS USED: The observable delegate executes custom logic (like logging price increases) whenever the property changes.

```
// 'by Delegates.observable' with business logic
var price: Int by Delegates.observable(0) {
    property, oldValue, newValue ->
    if (newValue > oldValue) {
        println("Price increased from $oldValue to $newValue")
    }
}
```

9. Vetoable Property

HOW 'by' IS USED: 'by Delegates.vetoable' delegates to a vetoable delegate that can reject changes. The lambda returns true to accept or false to reject the new value.

```
// 'by Delegates.vetoable' can reject changes
var max: Int by Delegates.vetoable(0) {
    property, oldValue, newValue ->
        newValue > oldValue // Only accept if increasing
}

max = 10 // Accepted (true)
max = 5  // Rejected (false), max stays 10
```

10. Vetoable with Validation

HOW 'by' IS USED: The vetoable delegate validates the new value before accepting it. Here it ensures age is within a valid range.

```
// 'by Delegates.vetoable' validates input
var age: Int by Delegates.vetoable(0) { _, _, new ->
    new >= 0 && new <= 150 // Must be valid age range
}
```

11. Storing Properties in a Map

HOW 'by' IS USED: 'by map' delegates property access to a Map. Getting the property looks up the key (property name) in the map.

```
// 'by map' looks up properties in a Map by name
class User(map: Map<String, Any?>) {
    val name: String by map // map["name"]
    val age: Int by map      // map["age"]
}

val user = User(mapOf(
    "name" to "John",
    "age" to 25
))

println(user.name) // Looks up "name" in map
```

12. Mutable Map Delegation

HOW 'by' IS USED: 'by map' with MutableMap allows both reading and writing. Setting a property updates the map.

```
// 'by map' with MutableMap supports read/write
class MutableUser(map: MutableMap<String, Any?>) {
    var name: String by map // Can get and set
    var age: Int by map
}

val map = mutableMapOf<String, Any?>(
    "name" to "Jane", "age" to 30
)

val user = MutableUser(map)
user.age = 31 // Updates map["age"] = 31
```

13. Map Delegation with Default Values

HOW 'by' IS USED: Properties delegated to a map can have their values pre-populated in the map constructor.

```
// 'by map' with configuration values
class Config(map: Map<String, Any?>) {
    val timeout: Int by map
    val retries: Int by map
}

val config = Config(mapOf(
    "timeout" to 5000, // Pre-populate map
    "retries" to 3
))
```

14. NotNull Property

HOW 'by' IS USED: 'by Delegates.notNull()' creates a non-nullable property that must be initialized before first use. It throws an exception if accessed before initialization.

```
import kotlin.properties.Delegates

// 'by Delegates.notNull()' requires initialization
var notNullValue: String by Delegates.notNull<String>()

// Must initialize before use
notNullValue = "Initialized"
println(notNullValue) // OK
// If accessed before init, throws exception
```

15. NotNull with Late Init Pattern

HOW 'by' IS USED: `notNull` delegate is useful for properties that can't be initialized in constructor but will be set before use.

```
// 'by Delegates.notNull()' for late initialization
class Service {
    var config: Config by Delegates.notNull()

    fun initialize(cfg: Config) {
        config = cfg // Must call this before using config
    }
}
```

16. Custom Property Delegate

HOW 'by' IS USED: `'by StringDelegate()'` uses a custom delegate class. The delegate must implement `getValue` and `setValue` operators.

```
import kotlin.reflect.KProperty

// Custom delegate implementing getValue/setValue
class StringDelegate {
    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): String {
        return "$thisRef, delegated '${property.name}'"
    }

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, value: String) {
        println("$value assigned to '${property.name}'")
    }
}

// 'by StringDelegate()' uses our custom delegate
var s: String by StringDelegate()
```

17. Custom Delegate with State

HOW 'by' IS USED: Custom delegates can maintain internal state. Each access to 'counter' increments and returns the count.

```
// Custom delegate with internal state
class Counter {
    private var count = 0 // Delegate's state

    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): Int {
        return count++ // Increment on each access
    }

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, value: Int) {
        count = value
    }
}

var counter: Int by Counter()
```

18. Custom Delegate with Validation

HOW 'by' IS USED: Custom delegate validates values in setValue. Only positive integers are stored; others become 0.

```
// Custom delegate validates on write
class PositiveInt {
    private var value: Int = 0

    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): Int = value

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, newValue: Int) {
        value = if (newValue > 0) newValue else 0 // Validate
    }
}

var positiveNumber: Int by PositiveInt()
```

19. Custom Delegate for Caching

HOW 'by' IS USED: Custom delegate implements caching logic. The initializer lambda runs only once, on first access.

```
// Custom delegate with lazy caching
class Cached<T>(val initializer: () -> T) {
    private var cached: T? = null

    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): T {
        if (cached == null) {
            cached = initializer() // Initialize once
        }
        return cached!!
    }
}

val cachedValue: String by Cached {
    expensiveOperation() // Called only once
}
```

20. Custom Delegate with Logging

HOW 'by' IS USED: Custom delegate adds logging to property access. Every get and set operation is logged.

```
// Custom delegate logs all access
class Logged<T>(private var value: T) {
    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): T {
        println("Getting ${property.name}")
        return value
    }

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, newValue: T) {
        println("Setting ${property.name} to $newValue")
        value = newValue
    }
}

var loggedProperty: String by Logged("initial")
```

21. Multiple Interface Delegation

HOW 'by' IS USED: 'by a' and 'by b' delegate different interfaces to different objects. Class C implements both A and B through delegation.

```
interface A { fun funcA() }
interface B { fun funcB() }

class AImpl : A {
    override fun funcA() = println("A")
}
class BImpl : B {
    override fun funcB() = println("B")
}

// 'by a, B by b' delegates each interface separately
class C(a: A, b: B) : A by a, B by b

val c = C(AImpl(), BImpl())
c.funcA() // Delegates to a
c.funcB() // Delegates to b
```

22. Selective Method Override with Delegation

HOW 'by' IS USED: 'by repo' delegates Repository interface, but we override load() to add caching logic while keeping save() delegated.

```
interface Repository {
    fun save()
    fun load()
}

class DefaultRepo : Repository {
    override fun save() = println("Saving")
    override fun load() = println("Loading")
}

// 'by repo' delegates, but override load() for caching
class CachedRepo(repo: Repository) : Repository by repo {
    override fun load() {
        println("Checking cache first")
        // Then could call delegated load if needed
    }
    // save() is still delegated
}
```

23. Delegation with Generic Interface

HOW 'by' IS USED: 'by container' delegates a generic interface Container to the container object.

```
interface Container<T> {
    fun add(item: T)
    fun get(): T
}

class ListContainer<T> : Container<T> {
    private val list = mutableListOf<T>()
    override fun add(item: T) { list.add(item) }
    override fun get(): T = list.last()
}

// 'by container' delegates generic interface
class LoggedContainer<T>(
    container: Container<T>
) : Container<T> by container
```

24. Delegation Chain

HOW 'by' IS USED: You can chain delegations. LoggedService delegates to a Service, which could itself delegate to another Service.

```
interface Service {
    fun execute()
}

class BaseService : Service {
    override fun execute() = println("Executing")
}

// 'by s' creates a delegation chain
class LoggedService(s: Service): Service by s {
    override fun execute() {
        println("Logging...")
    }
}

class CachedService(s: Service) : Service by s

// Can chain: CachedService -> LoggedService -> BaseService
```

25. Delegation with Companion Object

HOW 'by' IS USED: Companion objects can use delegation too. Here Component's companion delegates to DefaultFactory.

```
interface Factory {
    fun create(): String
}

class DefaultFactory : Factory {
    override fun create() = "Default"
}

// Instance delegates to factory parameter
class Component(factory: Factory) : Factory by factory {
    // Companion also uses delegation
    companion object : Factory by DefaultFactory()
}
```

26. Lazy with Custom Initializer

HOW 'by' IS USED: 'by lazy' can use any lambda, including function calls. The function is invoked only on first access.

```
// 'by lazy' with function call initializer
val config by lazy {
    loadConfigFromFile() // Called once on first access
}

fun loadConfigFromFile(): Config {
    // Complex initialization logic
    return Config()
}
```

27. Lazy Property in Data Class

HOW 'by' IS USED: Data classes can have lazy properties. Profile is fetched only when accessed.

```
// 'by lazy' in data class for expensive operations
data class User(val id: Int) {
    val profile: Profile by lazy {
        fetchProfileFromDB(id) // Fetch only when needed
    }
}
```

28. Multiple Lazy Properties

HOW 'by' IS USED: Each 'by lazy' property has its own independent initialization. Each is computed only when accessed.

```
// Multiple independent lazy properties
class Application {
    val database by lazy { Database() } // Init when used
    val cache by lazy { Cache() }      // Init when used
    val logger by lazy { Logger() }    // Init when used
}
```

29. Lazy with Exception Handling

HOW 'by' IS USED: The lazy initialization lambda can contain try-catch for error handling.

```
// 'by lazy' with error handling
val resource by lazy {
    try {
        loadResource()
    } catch (e: Exception) {
        defaultResource() // Fallback on error
    }
}
```

30. Lazy Circular Dependency Prevention

HOW 'by' IS USED: Lazy delegation helps break circular dependencies. Each service is created only when accessed, preventing initialization deadlock.

```
// 'by lazy' breaks circular dependencies
class ServiceA {
    val serviceB by lazy { ServiceB() } // Not created yet
}

class ServiceB {
    val serviceA by lazy { ServiceA() } // Not created yet
}

// Neither is created until accessed
```

31. Observable with Multiple Observers

HOW 'by' IS USED: The observable lambda can trigger multiple side effects when the property changes.

```
// 'by observable' with multiple side effects
var status: String by Delegates.observable("idle") {
    _, old, new ->
    notifyObservers(old, new)
    updateUI(new)
    logStatusChange(old, new) // All called on change
}
```

32. Observable Property Chain

HOW 'by' IS USED: Observable properties can trigger other observable properties, creating a chain reaction.

```
// Observable properties triggering each other
var temperature: Int by Delegates.observable(20) {
    _, _, new ->
    if (new > 30) {
        alarm = true // Triggers alarm's observable
    }
}

var alarm: Boolean by Delegates.observable(false) {
    _, _, new ->
    if (new) triggerAlert()
}
```

33. Observable with Conditional Updates

HOW 'by' IS USED: Observable can execute different logic based on the new value.

```
// 'by observable' with conditional logic
var value: Int by Delegates.observable(0) {
    prop, old, new ->
    if (new % 2 == 0) {
        println("Even: $new")
    } else {
        println("Odd: $new")
    }
}
```

34. Provider Delegate Pattern

HOW 'by' IS USED: provideDelegate is called during property creation to return the actual delegate. This allows validation or registration at declaration time.

```
// 'by ResourceProvider()' calls provideDelegate first
class ResourceProvider {
    operator fun provideDelegate(
        thisRef: Any?,
        prop: KProperty<*>
    ): ReadOnlyProperty<Any?, String> {
        checkProperty(prop) // Validate at declaration
        return object : ReadOnlyProperty<Any?, String> {
            override fun getValue(
                thisRef: Any?,
                property: KProperty<*>
            ) = "Resource"
        }
    }
}

val resource: String by ResourceProvider()
```

35. Provider with Registration

HOW 'by' IS USED: `provideDelegate` can register properties as they're declared, useful for tracking or validation.

```
// provideDelegate registers property names
class Registry {
    private val properties = mutableListOf<String>()

    operator fun provideDelegate(
        thisRef: Any?,
        prop: KProperty<*>
    ): ReadOnlyProperty<Any?, String> {
        properties.add(prop.name) // Register
        return object : ReadOnlyProperty<Any?, String> {
            override fun getValue(
                thisRef: Any?,
                property: KProperty<*>
            ) = prop.name
        }
    }
}
```

36. View Binding Delegate (Android)

HOW 'by' IS USED: `'by lazy'` defers `findViewById` until the view is accessed, avoiding unnecessary lookups.

```
// 'by lazy' for Android view binding
val textView: TextView by lazy {
    findViewById(R.id.textView) // Find view once
}
```

37. ViewModel Delegate (Android)

HOW 'by' IS USED: 'by viewModels()' is a custom Android delegate that manages ViewModel lifecycle.

```
// 'by viewModels()' Android lifecycle-aware delegate
private val viewModel: MyViewModel by viewModels()
// Handles ViewModel creation and lifecycle
```

38. Activity Result Delegate (Android)

HOW 'by' IS USED: 'by registerForActivityResult()' creates a delegate that manages activity result contracts.

```
// 'by registerForActivityResult' delegate
val launcher by registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()
) { result ->
    // Handle result through delegate
}
```

39. Preferences Delegate (Android)

HOW 'by' IS USED: Custom delegate for SharedPreferences that automatically reads/writes preference values.

```
// Custom SharedPreferences delegate
var theme: String by preferences.string(
    key = "theme",
    defaultValue = "light"
)
// Reads/writes to SharedPreferences automatically
```

40. ReadOnlyProperty Interface

HOW 'by' IS USED: Implementing ReadOnlyProperty interface allows custom read-only delegates. Only getValue is needed.

```
import kotlin.properties.ReadOnlyProperty
import kotlin.reflect.KProperty

// Custom read-only delegate
class FixedValue<T>(private val value: T) :
    ReadOnlyProperty<Any?, T> {
    override fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): T = value
}

val constant: Int by FixedValue(42)
// constant is read-only
```

41. ReadWriteProperty Interface

HOW 'by' IS USED: Implementing ReadWriteProperty interface allows custom read-write delegates. Both `getValue` and `setValue` are needed.

```
import kotlin.properties.ReadWriteProperty

// Custom read-write delegate
class Uppercase : ReadWriteProperty<Any?, String> {
    private var value = ""

    override fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ) = value

    override fun setValue(
        thisRef: Any?,
        property: KProperty<*>,
        newValue: String
    ) {
        value = newValue.uppercase() // Transform
    }
}

var text: String by Uppercase()
```

42. Delegation in Object

HOW 'by' IS USED: Objects (singletons) can use property delegation just like classes.

```
// 'by lazy' in singleton object
object Config {
    val timeout: Int by lazy {
        System.getProperty("timeout").toInt()
    }
}
```

43. Delegation in Companion Object

HOW 'by' IS USED: Companion objects can use delegation for singleton patterns within a class.

```
// 'by lazy' in companion for singleton pattern
class Database {
    companion object {
        val instance: Database by lazy {
            Database() // Thread-safe singleton
        }
    }
}
```

44. Top-Level Property Delegation

HOW 'by' IS USED: Top-level (global) properties can use delegation too.

```
// Top-level property with 'by lazy'
val globalConfig: Map<String, String> by lazy {
    loadConfiguration() // Global lazy init
}

fun loadConfiguration(): Map<String, String> {
    return mapOf("env" to "prod")
}
```

45. Local Variable Delegation

HOW 'by' IS USED: Even local variables in functions can use delegation (though less common).

```
// Local variable with 'by lazy'
fun process() {
    val result by lazy {
        expensiveComputation() // Only if result is used
    }

    if (shouldCompute) {
        println(result) // Computed here
    }
}
```

46. Delegation with Generics

HOW 'by' IS USED: Delegates can be generic, working with any type.

```
// Generic delegate
class Wrapper<T>(private val value: T) {
    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): T = value
}

// Works with any type
val wrappedInt: Int by Wrapper(10)
val wrappedString: String by Wrapper("Hello")
```

47. Delegation with Type Parameters

HOW 'by' IS USED: Delegates can use type parameters to convert between types.

```
// Delegate with type conversion
interface Converter<in T, out R> {
    fun convert(value: T): R
}

class StringToIntConverter : Converter<String, Int> {
    override fun convert(value: String) = value.toInt()
}

class ConvertedProperty<T, R>(
    private val converter: Converter<T, R>
) {
    private lateinit var original: T

    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): R = converter.convert(original) // Convert on get

    operator fun setValue(
        thisRef: Any?,
        property: KProperty<*>,
        value: T
    ) {
        original = value
    }
}
```

48. Nullable Property Delegation

HOW 'by' IS USED: Delegates work with nullable types too. Observable can track null values.

```
// Nullable property with observable
var nullableValue: String? by Delegates.observable(null) {
    _, old, new ->
    println("Changed from $old to $new")
}
```

49. Delegation with Inline Classes

HOW 'by' IS USED: Delegates can work with inline value classes for zero-overhead type safety.

```
// Delegate with inline class
@JvmInline
value class UserId(val value: Int)

class UserRepository {
    private val users = mutableMapOf<Int, User>()

    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): User? = users[property.name.hashCode()]
}

val currentUser: User? by UserRepository()
```

50. Combined Delegation Patterns

HOW 'by' IS USED: A single delegate can combine multiple patterns: validation, transformation, and custom logic all in one delegate.

```
// Delegate combining validation and transformation
class SmartProperty<T> {
    private var value: T,
    private val validator: (T) -> Boolean,
    private val transformer: (T) -> T
} {
    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): T = value

    operator fun setValue(
        thisRef: Any?,
        property: KProperty<*>,
        newValue: T
    ) {
        if (validator(newValue)) {
            value = transformer(newValue)
        }
    }
}
```

```
var email: String by SmartProperty(  
    value = "",  
    validator = { it.contains("@") },      // Validate  
    transformer = { it.lowercase() }      // Transform  
)
```

Quiz: Test Your Understanding of 'by' Keyword

Q1. What are the two main uses of the 'by' keyword in Kotlin?

A) Class delegation and property delegation

Q2. In 'class Derived(b: Base) : Base by b', what happens when you call a method on Derived?

A) The call is forwarded to object b

Q3. When using 'val x by lazy {}', when is the lambda executed?

A) On the first access to x, then cached

Q4. What's the difference between 'observable' and 'vetoable' delegates?

A) Observable notifies after change; vetoable can reject the change

Q5. In 'val name: String by map', how does it retrieve the value?

A) It looks up the property name ('name') as a key in the map

Q6. What must a custom delegate class implement to work with 'by'?

A) getValue operator (and setValue for mutable properties)

Q7. Can you override specific methods when using class delegation with 'by'?

A) Yes, you can override some methods while others remain delegated

Q8. What thread safety mode does 'by lazy' use by default?

A) SYNCHRONIZED - thread-safe with a lock

Q9. What happens if you access a property with 'by Delegates.notNull()' before initializing it?

A) It throws an `IllegalStateException`

Q10. In a custom delegate, what parameters does `getValue` receive?

A) `thisRef` (the object), `property` (property metadata)

Q11. Can multiple interfaces be delegated to different objects in the same class?

A) Yes, e.g., `'class C : A by a, B by b'`

Q12. What's the purpose of the `provideDelegate` operator?

A) To create/validate the delegate when the property is declared

Q13. Does 'by lazy' support nullable types?

A) Yes, you can use `'val x: String? by lazy { null }'`

Q14. What's the difference between 'by map' with `Map` vs `MutableMap`?

A) `Map` is read-only; `MutableMap` allows property assignment

Q15. Can delegation be used with top-level properties?

A) Yes, e.g., `'val config by lazy { loadConfig() }'`

Q16. In 'by `Delegates.observable`', when is the lambda called?

A) After the property value has already changed

Q17. What does `LazyThreadSafetyMode.PUBLICATION` mean?

A) Multiple threads may initialize, but only first result is used

Q18. Can you chain multiple delegations together?

A) Yes, delegate to an object that itself uses delegation

Q19. What interfaces from `kotlin.properties` help create custom delegates?

A) `ReadOnlyProperty` and `ReadWriteProperty`

Q20. Why use 'by lazy' instead of direct initialization?

A) Defers expensive computation until actually needed

Q21. Can companion objects use property delegation?

A) Yes, commonly used for singletons with 'by lazy'

Q22. What's a common use case for 'by Delegates.vetoable'?

A) Input validation - rejecting invalid values

Q23. In class delegation, do delegated methods have access to the delegating class's members?

A) No, they only access the delegate object's members

Q24. Can local variables inside functions use delegation?

A) Yes, though it's less common than class properties

Q25. What happens if a 'by lazy' initialization lambda throws an exception?

A) The exception is propagated, and the value remains uninitialized

Q26. How does 'by map' determine which map key to use?

A) It uses the property name as the key

Q27. Can you use delegation with generic types?

A) Yes, delegates can be generic and work with any type

Q28. What's the primary benefit of class delegation over inheritance?

A) Composition over inheritance - more flexible, less coupling

Q29. Is it possible to have both 'by lazy' and a custom setter?

A) No, lazy creates a read-only property (val)

Q30. What's a practical use of observable delegates?

A) Updating UI or triggering side effects when data changes